broadcast where you expect it to go.

That's the client side. On the server side, we need to make sure that whenever a Favorite object is created or destroyed, we send the appropriate message down the appropriate ActionCable.

Perhaps surprisingly, we make that declaration in the Favorite model object. I say *surprisingly* because sending something over an ActionCable is a view responsibility and not typically seen as a responsibility of the model. On the other hand, the turbo-rails development team has gone to some trouble to make it not only possible but also relatively uncomplicated to allow you to connect the ActionCable via the model. (Strictly speaking, you don't have to connect via the model, but more on that in a second.)

Let's look at the code and then discuss the architecture implications.

Here's what our Favorite ActiveRecord class looks like with the Turbo Stream support enabled:

chapter_09/01/app/models/favorite.rb

```
class Favorite < ApplicationRecord
  belongs_to :user
  belongs_to :concert

  after_create_commit -> do
    broadcast_append_later_to(user, :favorites, target: "favorite-concerts")
  end

  after_destroy_commit -> { broadcast_remove_to(user, :favorites) }
end
```

The code here uses ActiveRecord callbacks to trigger messages to the ActiveCable channel. *ActiveRecord callbacks* are hooks that are invoked by Rails at various times in the life cycle of the object.[69] The callbacks typically take a method name or a Ruby lambda as an argument.

Let's take this one callback at a time. The first callback is after_create_commit. It is